

AlgoBench : 알고리즘 벤치마킹 프로그램

프로젝트 참여자

Submission Roster

구분	학과	학번	이름
설계	AI 컴퓨터 공학부 인공지능 전공	202311516	권창민

목차

1. 프로젝트 개요

1.2 개발 배경 및 개인적 의도

2. 요구사항 분석

2.2 기능 요구사항

2.4 제약 사항 및 범위

3.1 자체 포맷 문제 로드

3.3 멀티스레드 기반 비동기 채점 엔진

4. 객체지향 설계 방향

5.1. 전체 구조 요약

5.3. 문제/테스트케이스 도메인

5.5. 결과/로깅 구조

1.1 프로젝트 정의

1.3 기대 효과

2.1 사용자 및 사용 시나리오

2.3 비기능 요구사항

3. 핵심 기능

3.2 다형성 기반 다언어 코드 로딩 및 실행

3.4 벤치마킹 결과 수집 및 리포팅

5. 클래스 다이어그램

5.2. 채점 실행 흐름

5.4. 솔루션 실행 전략

5.6. 전체 통합 구조

1. 프로젝트 개요

1.1 프로젝트 정의

AlgoBench는 알고리즘 문제와 테스트 케이스를 로컬 파일로 관리하고, 여러 개의 풀이 코드를 동일한 조건에서 실행하여 정답 여부와 실행 시간을 비교하는 로컬 알고리즘 벤치마킹 프로그램이다.

본 프로그램은 온라인 저지처럼 대규모 사용자 제출을 처리하는 플랫폼이 아니라, 개인 또는 스터디 그룹이 직접 만든 문제 세트에 대해 여러 풀이 방식을 검증하고 성능을 비교하기 위한 학습용 로컬 저지 도구를 목표로 한다.

1.2 개발 배경 및 개인적 의도

본 프로젝트는 최근 국내 최대 알고리즘 채점 플랫폼인 Baekjoon Online Judge(BOJ)의 서비스 종료 이슈에서 아이디어를 얻었다. 특정 외부 플랫폼에 전적으로 의존하는 기존의 학습 방식은 해당 서비스의 장애나 중단 시, 스터디 커리큘럼이 완전히 마비될 수 있다는 치명적인 리스크를 안고 있다.

나는 알고리즘 풀이를 단순히 정답을 맞히는 과정으로만 보지 않고, 같은 문제를 다양한 방식으로 해결했을 때 실행 시간, 실패 원인, 안정성이 어떻게 달라지는지 비교하는 과정까지 학습의 일부라고 생각했다.

따라서 AlgoBench는 외부 서비스 없이도 문제 입력, 기대 출력, 풀이 실행, 결과 비교, 성능 기록을 한 흐름으로 수행할 수 있는 독립적인 실험 환경을 제공하는 것을 목적으로 한다.

1.3 기대 효과

- 외부 온라인 저지 플랫폼에 의존하지 않고 로컬 환경에서 알고리즘 풀이를 검증할 수 있다.
- 동일한 문제에 대해 여러 풀이 방식의 실행 시간과 실패 원인을 비교할 수 있다.
- 문제 로딩, 코드 실행, 정답 비교, 결과 리포팅을 분리하여 객체지향 설계 원칙을 실제 프로그램 구조에 적용할 수 있다.
- 향후 새로운 언어 실행 방식, 새로운 출력 비교 정책, 새로운 리포트 형식을 확장하기 쉬운 구조를 확보할 수 있다.

2. 요구사항 분석

2.1 사용자 및 사용 시나리오

AlgoBench의 주요 사용자는 알고리즘을 학습하는 개인 학습자 또는 스터디 그룹이다. 사용자는 직접 작성한 문제 파일과 여러 풀이 코드를 준비한 뒤, 프로그램을 실행하여 각 풀이가 모든 테스트 케이스를 통과하는지와 실행 시간이 어느 정도인지 확인한다.

대표적인 사용 흐름은 다음과 같다.

- 사용자는 문제 제목, 제한 시간, 테스트 케이스가 포함된 로컬 문제 파일을 준비한다.

- 사용자는 Java 클래스/JAR 또는 외부 실행 파일 형태의 풀이 코드를 준비한다.
- 프로그램은 문제 파일을 읽고, 각 풀이를 동일한 테스트 케이스에 대해 실행한다.
- 프로그램은 실제 출력과 기대 출력을 비교하여 테스트 케이스별 통과 여부를 판단한다.
- 프로그램은 풀이별 성공 여부, 실패 위치, 실행 시간, 에러 메시지를 콘솔 또는 CSV로 기록한다.

2.2 기능 요구사항

ID	요구사항	설명
FR-01	문제 파일 로드	자체 정의한 문제 파일에서 제목, 제한 시간, 테스트 케이스를 읽어온다.
FR-02	테스트 케이스 관리	하나의 문제는 여러 입력과 기대 출력 쌍을 가진다.
FR-03	풀이 코드 실행	Java 클래스/JAR 또는 외부 실행 파일을 공통 인터페이스로 실행한다.
FR-04	정답 판정	실제 출력과 기대 출력을 비교하여 테스트 케이스별 통과 여부를 판단한다.
FR-05	실행 시간 측정	각 테스트 케이스 및 전체 풀이의 실행 시간을 측정한다.
FR-06	병렬 채점	여러 풀이를 스레드 풀을 이용해 비동기적으로 채점한다.
FR-07	결과 리포팅	풀이별 성공 여부, 실패 테스트 케이스, 실행 시간, 에러 메시지를 CSV 또는 콘솔로 출력한다.

2.3 비기능 요구사항

ID	요구사항	설명
NFR-01	독립 실행성	네트워크, DB, 외부 서버 없이 로컬 환경에서 실행되어야 한다.
NFR-02	Pure Java	Java SE 표준 라이브러리를 중심으로 구현한다.
NFR-03	확장성	새로운 실행 방식이나 채점 기준을 기존 코드 수정 없이 추가할 수 있어야 한다.
NFR-04	안정성	무한 루프나 런타임 에러가 전체 프로그램을 중단시키지 않아야 한다.
NFR-05	스레드 안전성	병렬 채점 중 결과 저장 과정에서 데이터가 섞이지 않아야 한다.
NFR-06	가독성	결과 리포트는 사용자가 실패 원인과 성능 차이를 쉽게 파악할 수 있어야 한다.

2.4 제약 사항 및 범위

- 본 프로젝트는 Java 프로그래밍 수업 프로젝트의 범위에 맞추어 Java SE 표준 라이브러리 중심으로 구현한다.

- 온라인 서비스 형태가 아니라 로컬 파일 시스템 기반의 Stand-alone 프로그램을 목표로 한다.
- 대규모 채점 서버, 사용자 계정 관리, 웹 UI, 데이터베이스 저장 기능은 이번 프로젝트 범위에서 제외한다.
- 핵심 목표는 알고리즘 풀이를 로컬에서 실행하고, 정답 여부와 성능 정보를 비교할 수 있는 최소한의 벤치마킹 흐름을 완성하는 것이다.

3. 핵심 기능

3.1 자체 포맷 문제 로드

- **커스텀 데이터 파싱** : 자체적으로 정의한 텍스트 포맷을 파싱하여 문제 정보 읽어오기.
- **메타데이터 및 테스트 케이스 관리** : 문제 제목, 시간 제한 등의 메타데이터와 다수의 입력(`stdin`) | 기대 출력(`stdout`) 쌍을 포함하는 테스트 케이스를 메모리에 객체화하여 관리합니다.

3.2 다형성 기반 다언어 코드 로딩 및 실행

- **자바 플러그인 동적 로딩 (`URLClassLoader`)** : 스터디원이 제출한 자바 컴파일(.jar 또는 .class)을 런타임에 동적으로 읽어와 Reflection을 통해 실행됩니다.
- **외부 프로세스 연동 (`ProcessBuilder`)** : C, C++, Python 등 타 언어로 작성된 실행 파일이나 스크립트를 독립된 OS 프로세스로 띄워 실행합니다.
- **표준 스트림 통신** : 런타임 환경에 구애받지 않고, 공통 인터페이스(`Solution`)을 통해 표준 입력 스트림으로 테스트 케이스를 주입하고 표준 출력 스트림으로 결과를 반환 받습니다.

3.3 멀티스레드 기반 비동기 채점 엔진

- **스레드 풀 활용** : `ExecutorService` 를 활용하여 여러 스터디원의 코드를 동시에 병렬로 채점하여 벤치마킹 소요 시간을 단축합니다.
- **독립 격리 실행 및 타임아웃** : 각 채점 작업(`GradingTask`)는 독립된 스레드에서 실행되며, 무한 루프 등 악의적이거나 비효율적인 코드로부터 코어 엔진을 보호하기 위해 엄격한 시간 초과 제어 로직을 포함합니다.
- **유연한 채점 방법** : 공백 무시, 대소문자 무시 등 다양한 채점 기준을 적용할 수 있도록 비교 로직(`OutputComparator`)을 모듈화하여 정답 여부를 판별합니다.

3.4 벤치마킹 결과 수집 및 리포팅

- **정밀한 성능 측정** : 자바 표준 API(`java.time.Duration`)를 활용하여 각 코드의 입출력 및 연산 소요 시간을 정밀하게 측정합니다.
- **에러 로깅** : 오답 시 몇 번째 테스트 케이스에서 실패했는지, 혹은 런타임 에러(`stderr`)가 발생했는지 상세 내역을 수집합니다.
- **CSV 포맷 리포트 생성** : 수집된 벤치마킹 결과 데이터를 단일 책임 원칙(SRP)에 따라 전용 포맷터(`ResultFormatter`)를 거쳐 가독성 높은 CSV 파일 형태로 저장합니다.

4. 객체지향 설계 방향

요구사항 분석 결과, AlgoBench는 문제 로딩, 풀이 실행, 정답 비교, 결과 기록이라는 서로 다른 책임을 가진다. 따라서 각 책임을 독립적인 클래스로 분리하고, 실행 방식과 출력 방식처럼 변경 가능성이 높은 부분은 인터페이스로 추상화하였다.

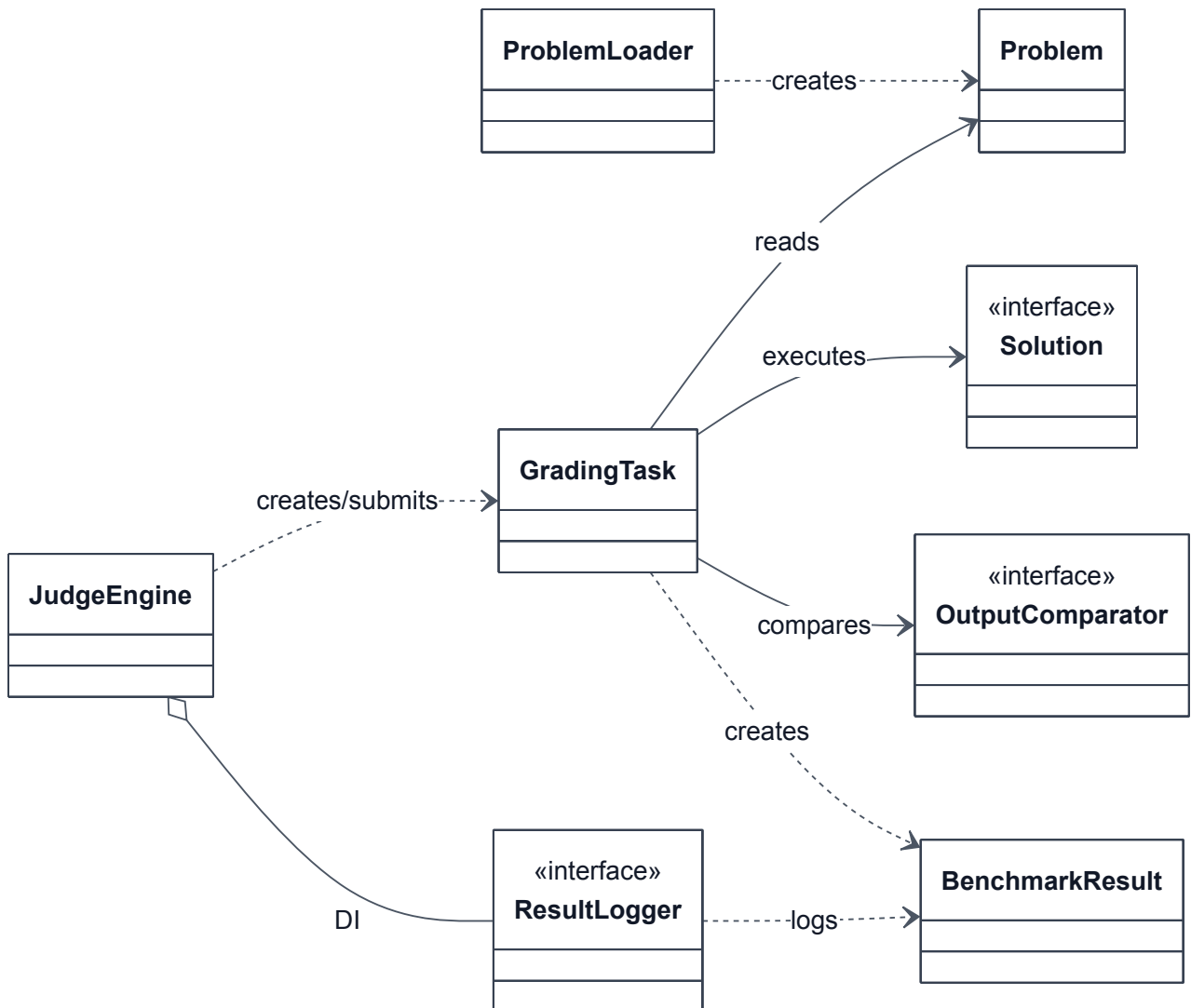
특히 `Solution`, `OutputComparator`, `ResultLogger`, `ResultFormatter` 는 확장 가능성이 높은 지점이므로 인터페이스로 설계하였다. 이를 통해 새로운 언어 실행 방식, 새로운 정답 비교 정책, 새로운 리포트 출력 형식을 기존 채점 엔진을 수정하지 않고 추가할 수 있다.

- `ProblemLoader` 는 로컬 문제 파일을 읽어 `Problem` 객체를 생성한다.
- `Problem` 과 `TestCase` 는 알고리즘 문제와 테스트 데이터를 표현하는 도메인 객체이다.
- `Solution` 은 풀이 코드 실행 방식을 추상화하여 Java 실행 방식과 외부 프로세스 실행 방식을 동일하게 다룬다.
- `GradingTask` 는 하나의 풀이를 하나의 문제에 대해 채점하는 실행 단위이며, `Callable` 을 통해 병렬 채점 구조와 연결된다.
- `BenchmarkResult` 와 `TestCaseResult` 는 채점 결과를 데이터 객체로 분리하여 결과 분석과 리포팅을 쉽게 만든다.
- `ResultLogger` 와 `ResultFormatter` 는 결과 출력 방식과 포맷 변환 책임을 분리한다.

5. 클래스 다이어그램

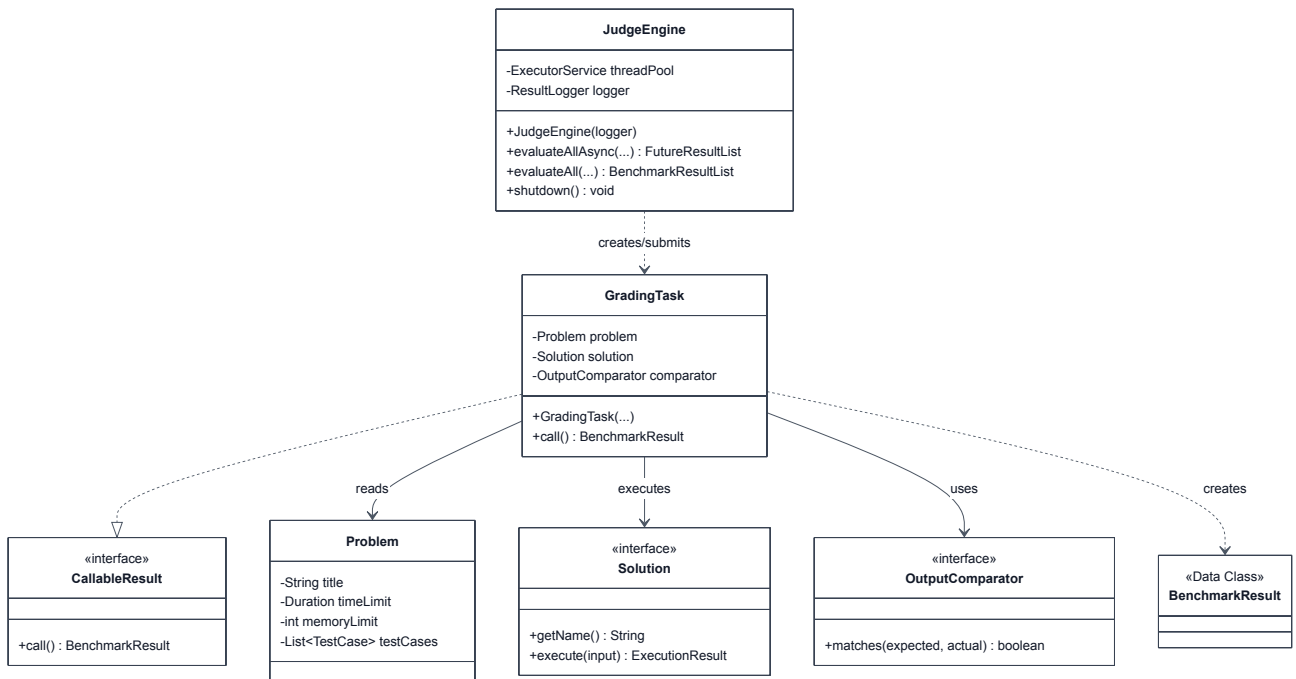
5.1. 전체 구조 요약

출력용 확대 다이어그램



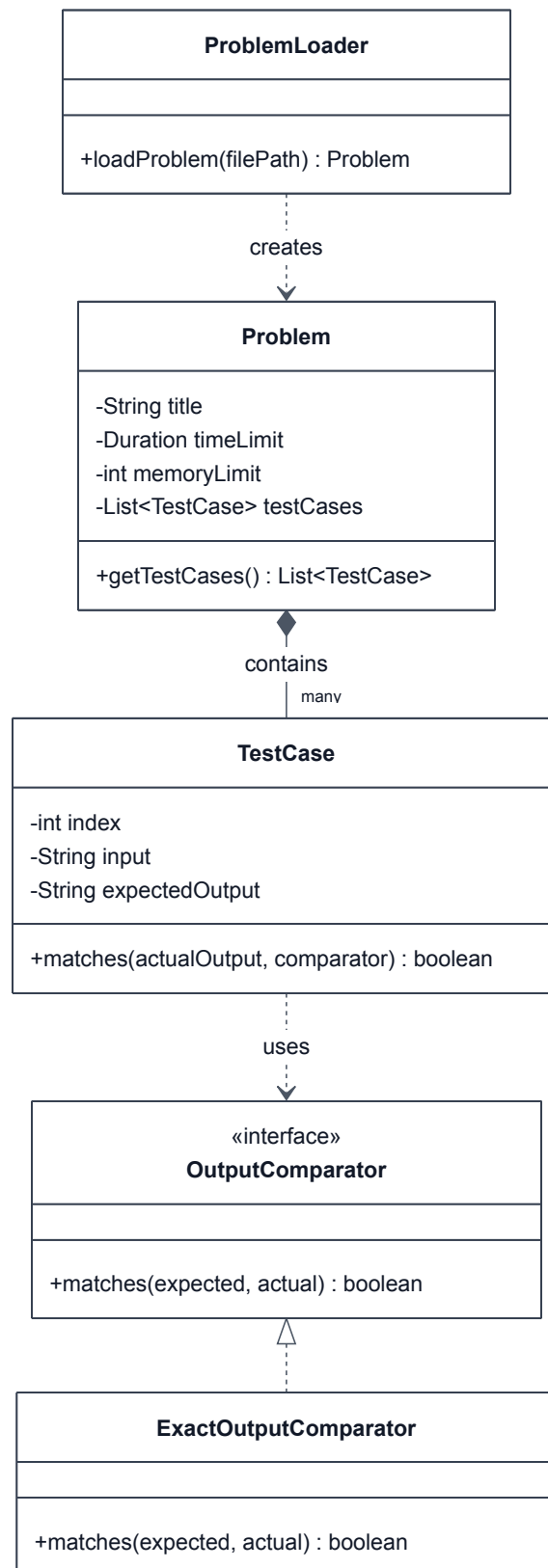
5.2. 채점 실행 흐름

출력용 확대 다이어그램



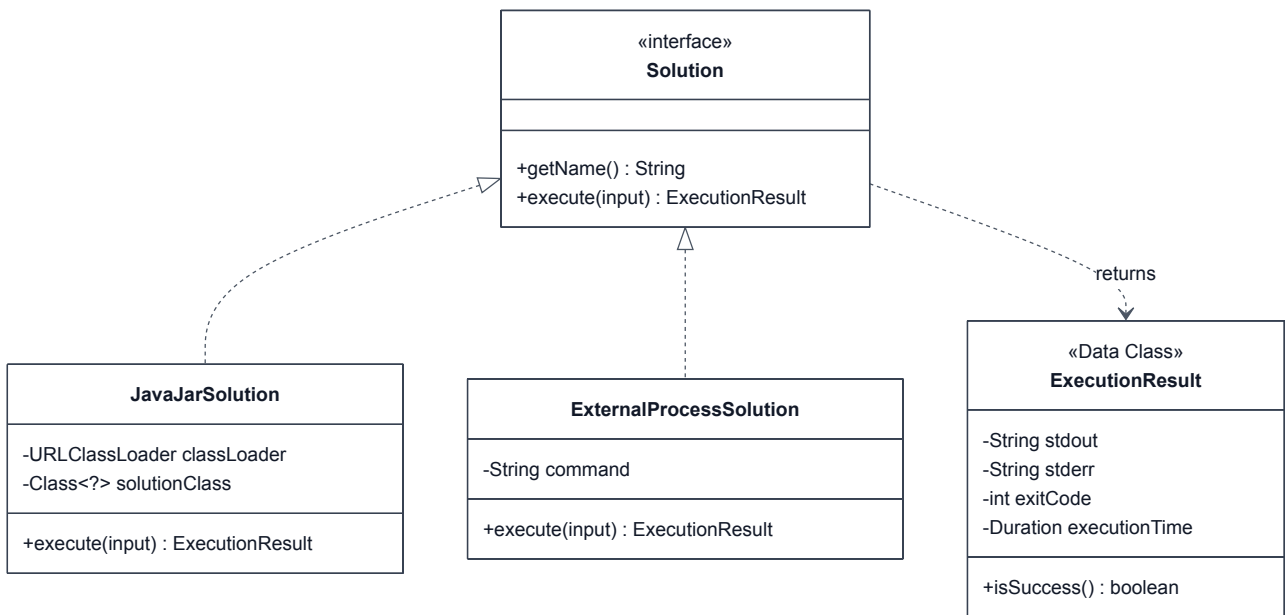
5.3. 문제/테스트케이스 도메인

출력용 확대 다이어그램



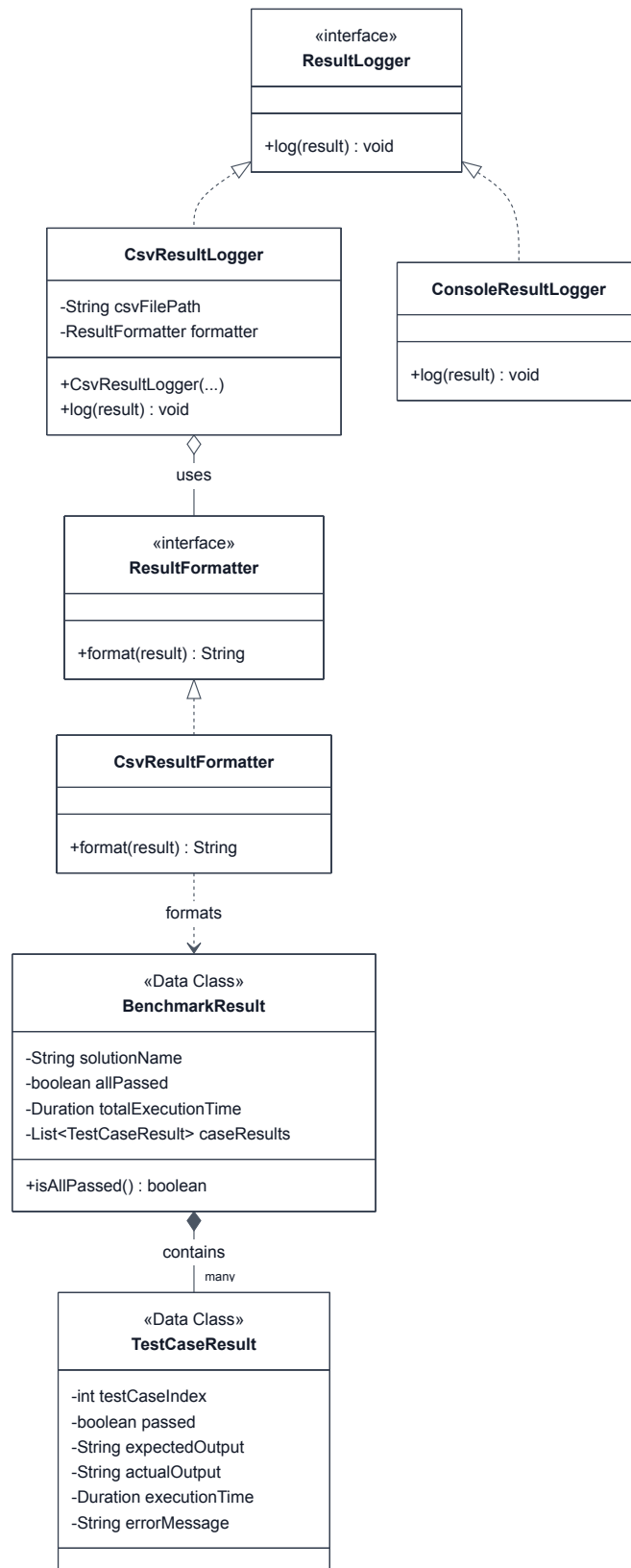
5.4. 솔루션 실행 전략

출력용 확대 다이어그램



5.5. 결과/로깅 구조

출력용 확대 다이어그램



5.6. 전체 통합 구조

부록용 전체 구조

